Assignment 1: Complexity and Sorting

Description

In this assignment, you'll work in groups to create an application for performing experimental analysis (benchmarking) of a list of objects using various sorting algorithms.

Warning: This project is significantly larger and more complex than the assignments in your previous semesters, do NOT wait until the last week of the deadline to start!

Equipment and Materials

For this assignment, you will need:

- Eclipse and Java 21

Note: If your submission does not meet these requirements, you will receive a grade of zero!

Instructions

This assignment consists of three parts, to be completed **outside** of class time. See the course outline and Brightspace for due dates. Complete this assignment with your assigned group.

Part A: Create a Sorting Application (90 marks)

Using the specifications below, create an application to sort objects according to a specific property. You'll then implement and perform experimental analysis (benchmarking) on six sorting algorithms:

- Bubble
- Insertion
- Selection
- Merge
- Quick using the "Median-of-3" approach for pivot selection
- A sorting algorithm of your choice*

Note: Think of it as writing libraries that perform sorting in descending order instead of the regular ascending order that already exist!

*You must research the sorting algorithm of your choice and ensure that it's **significantly different** from the other five. This must be an algorithm that does complete sorting the test files within a reasonable amount of time! (e.g. no bogosort!) Include a detailed description of this sort's algorithm and a complexity analysis outlined in the *Submission Deliverables* section of this document.

© 2026, Southern Alberta Institute of Technology

Note: If you're unsure of the algorithm you're allowed to choose, please discuss it with your instructor!

Part B: Complete an Application Evaluation as a Group (5 marks)

After completing your sorting application, check your work against the provided marking criteria. Your instructor will refer to your group's self-evaluation when grading the assignment and will provide further feedback and grade adjustments as needed. Your instructor is responsible for awarding the group's final grade.

1. Open the Marking Criteria document (MarkingCriteria_Assignment1.docx) and save a copy with your group's name.
2. As a group, discuss how well you met each criterion and assign yourselves a mark for each row in the table. You may include a short, point form, explanation for your mark in the Notes column.

   Note: This is an opportunity for you to identify any missing pieces according to the marking criteria, make sure you do this step diligently to avoid losing any marks that is clearly stated in this document.
3. Save this file for submission to Brightspace along with your completed code.

Part C: Complete a Peer Evaluation Assesment (5 marks)

Each student must also complete a peer assessment of themselves and their group members, which will be used to determine the final calculated grades of this assignment for all group members. Your instructor will provide further submission details in class.

Assignment Specifications

Read the specifications very carefully. If you are uncertain about any of the requirements, discuss it with your instructor.

1. Import the assignment1StartingCode project into Eclipse.
2. Create an abstract class in Java that represents a product type with 2 attributes:
   - Original Price
   - Updated Price
3. Your application will read a text file (specified at the command line via the -f option) of random objects and adds them to an array (not ArrayList!).

   Notes:

   - The value in the first line of the data file contains the number of objects in that file.
     - This will determine the size of your array.

© 2026, Southern Alberta Institute of Technology

- Each subsequent line in the data file will contain the 2 attributes that you will need to generate the corresponding object.
- An object in the file is represented as follows on each line (the values are separated by spaces):
    - One of the following product types: “Jeans", "TShirts", "Skirts", "Dresses", "Shorts", "Jackets", "Shoes”
    - Each product type should be a subclass of the abstract class from step 2.
    - Each object is followed by 2 double values representing the original price and another double value representing updated price.

      e.g., Shorts 5.25 23.36 or Jeans 7.83 71.4

4. This base class should also have an abstract method `calcRealDiff()` to calculate the difference between the 2 attributes with adjustments based on the type of product:
    - Adjustment * Absolute value of (Original Price - Updated Price)

Product Type	Adjustment
Jeans	0.5
TShirts	0.2
Skirts	0.1
Dresses	0.6
Shorts	0.3
Jackets	0.7
Shoes	0.4

5. Implement the **compareTo()** method of the Comparable interface in the base class to compare two objects using the return value of `calcRealDiff()`.
6. Implement 2 separate external classes using the **compare()** method of the Comparator interface to compare two objects by their original price and updated price.

   Note: Your implementations of these interfaces MUST adhere to the contract! You can NOT violate the rules of the interface as specified. (i.e. you can not switch the positive vs. negative return values!)

7. Your application should then invoke the sort methods in the utilities package to re-arrange the objects according to the compare type from the **largest to smallest** (descending order) using the chosen sorting algorithm:
    - The compare type will be provided as input from the command line to your program: **o** for original value, **u** for updated value, and **d** for the calculated difference via the -t option.
    - The choice of sorting algorithm will be provided as input from the command line to your program: **b** for bubble sort, **s** for selection sort, **i** for insertion sort, **m** for merge sort, **q** for quicksort and **z** for your choice of sorting algorithm.

© 2026, Southern Alberta Institute of Technology

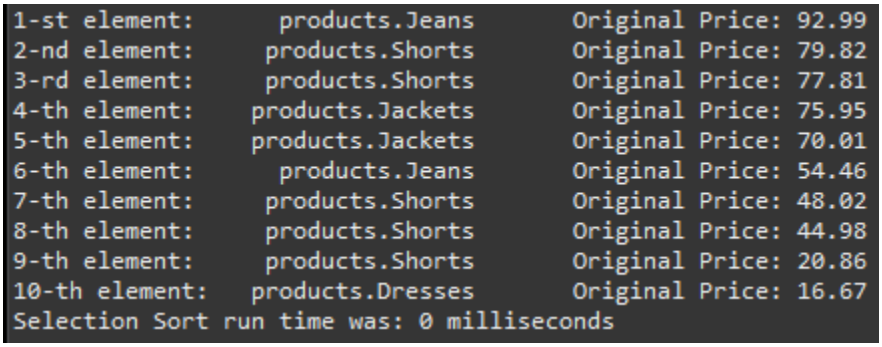
Note: Sorting in ascending order and then reversing the array is NOT acceptable! Your sorting algorithm must use the comparable and comparators correctly to re-arrange the objects.

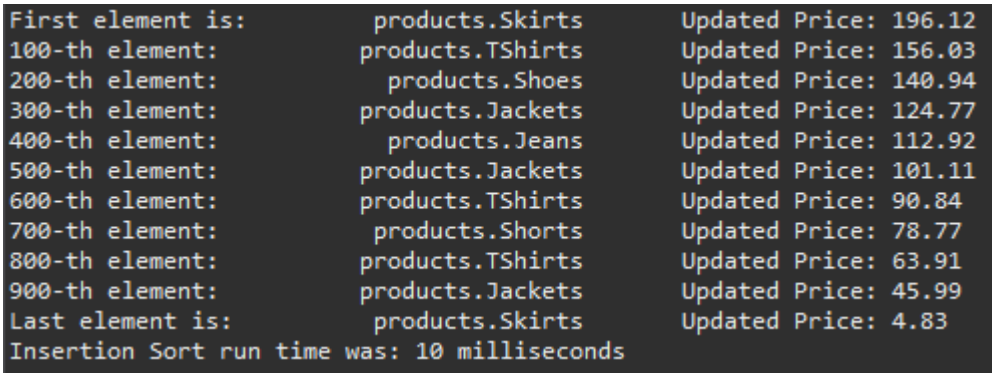
8. Your application should print the time that it took to sort the collection of objects for each of the six sorting algorithms, including a measurement unit (e.g., milliseconds). This should ONLY include the time for the sorting algorithm to run – not including the time to read the input file, generate the objects, or display the objects' data to the screen.

   Note: Have this benchmarking done outside of the sorting algorithms such that the sorting code can be executed without this explicit output.

9. The program should also display to the console:

   - If there are 10 objects or less – display all objects in sorted order
   - If there are 1000 objects or less – display the first sorted value and last sorted value, and every hundredth value in between
   - Otherwise, display the first sorted value and last sorted value, and every thousandth value in between, for example:





Note: These images are provided simply to give you an example of what the output would look like – your program output should be similar but does NOT have to match the exact formatting as long as the relevant information are shown.

© 2026, Southern Alberta Institute of Technology

Note: You can leave the calculated difference values as unbounded decimal.

```
First element is:        products.Jackets       Real Difference: 69.965
1000-th element:         products.Jackets       Real Difference: 53.507999999999996
2000-th element:           products.Jeans       Real Difference: 45.349999999999994
3000-th element:           products.Jeans       Real Difference: 39.045
4000-th element:         products.Dresses       Real Difference: 34.482
5000-th element:           products.Shoes       Real Difference: 29.944000000000006
6000-th element:          products.Shorts       Real Difference: 26.765999999999995
7000-th element:          products.Shorts       Real Difference: 23.604000000000003
8000-th element:          products.Shorts       Real Difference: 20.385
9000-th element:         products.TShirts       Real Difference: 18.198000000000004
10000-th element:        products.TShirts       Real Difference: 16.096
11000-th element:          products.Shoes       Real Difference: 13.956000000000001
12000-th element:        products.TShirts       Real Difference: 11.666000000000004
13000-th element:        products.TShirts       Real Difference: 9.642000000000001
14000-th element:         products.Skirts       Real Difference: 8.358
15000-th element:         products.Skirts       Real Difference: 7.022
16000-th element:          products.Shoes       Real Difference: 5.7
17000-th element:         products.Skirts       Real Difference: 4.304
18000-th element:         products.Skirts       Real Difference: 2.986
19000-th element:         products.Shorts       Real Difference: 1.704000000000002
20000-th element:          products.Jeans       Real Difference: 0.3200000000000003
Last element is:         products.TShirts       Real Difference: 0.0
Quick Sort run time was: 27 milliseconds
```

10. You will need to export your application to a runnable JAR file. To run this executable program, the user should be able to enter the following via the command prompt using the following case insensitive format:



`java -jar Sort.jar -f<file_name> -t<compare_type> -s<sort_alg>`

- -f or -F followed by file name (the file name and path) with no spaces in-between
- -t or -T followed by o, u or d with no spaces in-between
- -s or -S followed by b, s, i, m, q or z with no spaces in-between

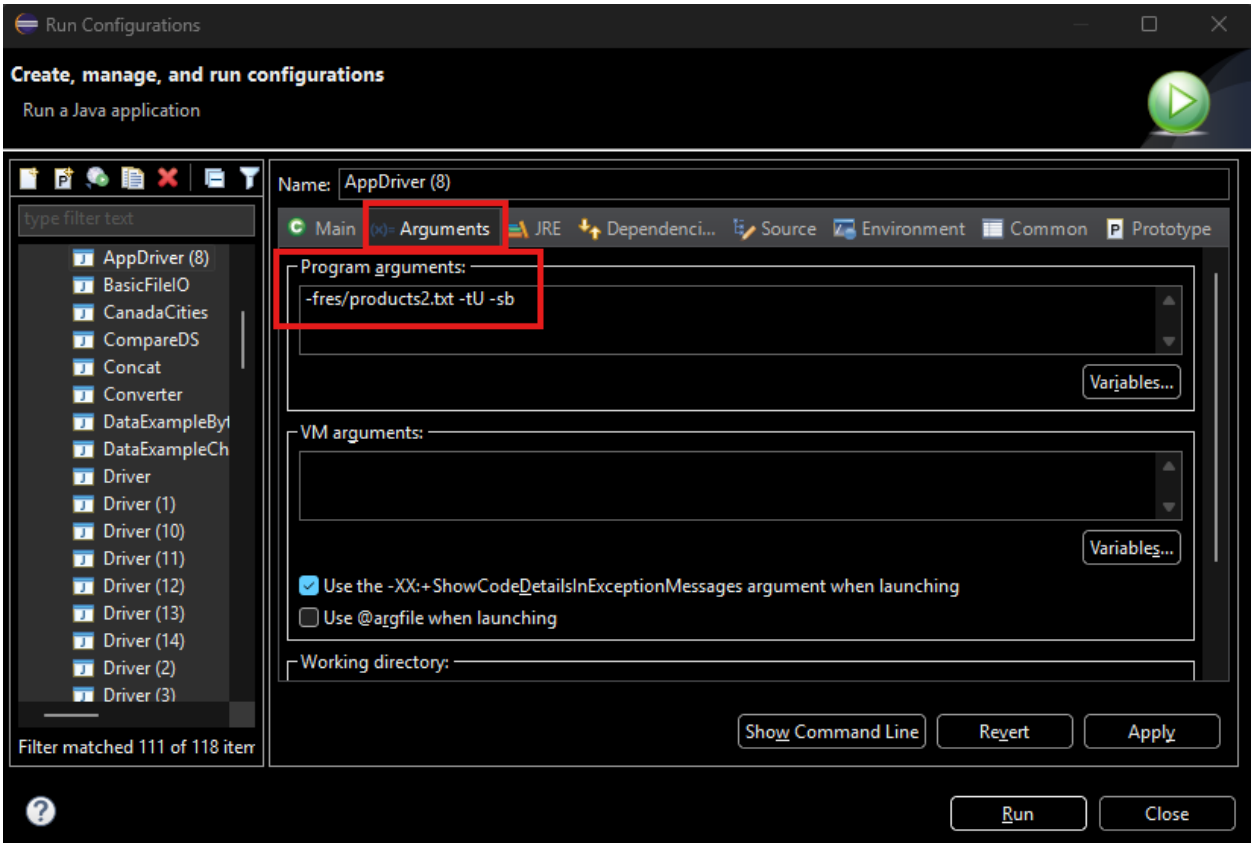
The program must be able to handle these arguments no matter the order (order insensitive). The following are some examples of valid inputs:

- `java -jar Sort.jar -fproducts1.txt -To -Sb`
- `java -jar Sort.jar -tu -sQ -f"res\products1.txt"`
- `java -jar Sort.jar -tD -F"C:\temp\products1.txt" -sZ`

Note: You can assume the user will provide the correct absolute or relative path to the required file as shown in the examples above.

© 2026, Southern Alberta Institute of Technology

You can test this input in Eclipse using the “Run Configurations” tool under the “Arguments” tab:



11. If the user enters an incorrect command line argument according to the rules above, the program should display a meaningful message to help the user in correcting the error.
12. You will want to first test your sorting algorithms starting with the smallest data set (products1.txt) to check the results.
13. Make sure your code works with the other test data! (i.e., products2.txt and products3.txt)
    - Your instructor will use a different test data file to evaluate your application.

© 2026, Southern Alberta Institute of Technology

Submission Deliverables

Note: This is a group assessment, so only one submission is required per group. If there are multiple submissions, your instructor will only evaluate the last submission. See Brightspace for the exact due date and time.

Your group's submission should include a zipped folder named as the assignment number and your group name (e.g., A1Zelda.zip). The zipped folder should contain the following items:

1) An **executable** Java Archive file (.jar) for your sorting application called **Sort.jar**.
2) A zipped file of the folder containing the complete Eclipse project directory.
    - Using File Explorer, navigate to your project folder inside your Eclipse workspace, right-click and select "Compress to… ZIP file".
3) A file (in any document format, (e.g., text, word, pdf etc.) that provides a detailed description of the sorting algorithm of your choice along with its complexity analysis.
    - Your analysis must include the steps of the algorithm in **pseudocode**, with a **counting analysis** (the number of operations performed in each step).

    Note: Make sure to write this in your own words and NOT copy or rephrase from another source.

4) The completed Marking Criteria document containing your group's evaluation of your application. All group members must be present when completing this document and it acts as your "sign-off" (approval) of the assignment submission.
5) Complete the Peer Evaluation Assessment as instructed by your instructor. Examples of how this evaluation will be used to determine your final grade for this assignment can be found on D2L → Unit 0: Course Information → Individual Grade Calculations for Group Assignments.pdf.

No late assignments or resubmissions will be accepted; you are responsible for the correctness of your deliverables.

© 2026, Southern Alberta Institute of Technology